



Ambiente de Desenvolvimento em Python



Configuração do Ambiente

Nesta semana, o foco será garantir que todos tenham um ambiente de desenvolvimento Python configurado corretamente para a prática.



Ferramentas Essenciais

Exploraremos o Jupyter Lab, Google Colab e Visual Studio Code, ferramentas cruciais para a prática de programação.



Fundamentos de Python

Também revisaremos alguns comandos básicos de Python, reforçando a base para os próximos conteúdos.

Ecosystema de Ferramentas



Python (≥ 3.6)

A linguagem de programação fundamental que utilizaremos em nosso desenvolvimento.



Pip

O gerenciador de pacotes essencial para instalar e atualizar bibliotecas Python em seus projetos.



Virtualenv

Ferramenta para criar ambientes virtuais, isolando as dependências de cada projeto e evitando conflitos.



IPython

Um terminal interativo mais avançado que o console padrão do Python, oferecendo recursos aprimorados.

Ecossistema de Ferramentas

A seguir, uma visão detalhada das ferramentas e bibliotecas essenciais para o seu ambiente de desenvolvimento Python.

Ambientes e IDEs



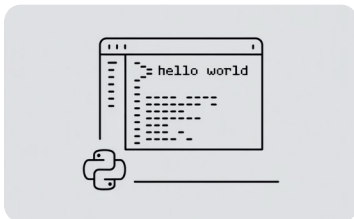
Jupyter Lab

Ambiente web interativo para criar e executar notebooks, ideal para análise de dados e prototipagem.



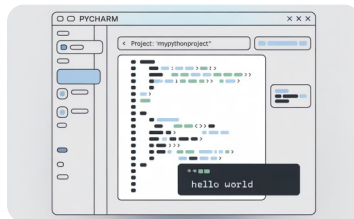
Google Colab

Uma versão online do Jupyter, que oferece um ambiente pronto e gratuito com acesso a GPUs e TPUs.



VS Code

Um editor de código leve e poderoso, amplamente personalizável para desenvolvimento Python.



PyCharm

Uma IDE completa e inteligente, projetada especificamente para o desenvolvimento profissional em Python.

Bibliotecas Essenciais



NumPy

Fundamental para computação numérica, oferece suporte a arrays e matrizes de alta performance.



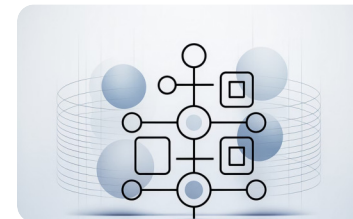
Pandas

Usado para manipulação e análise de dados, com estruturas como DataFrames para facilitar o trabalho.



Matplotlib & Seaborn

Bibliotecas poderosas para criação de visualizações de dados estáticas e estatísticas.



Scikit-learn & Plotly

Scikit-learn para aprendizado de máquina e Plotly para gráficos interativos e dashboards.

Instalação e Configuração

Antes de começar a programar, você precisa configurar seu ambiente de desenvolvimento:

1 Python

Faça o download e instale a versão mais recente do Python através do site oficial ou assista ao tutorial: [Tutorial de Instalação do Python](#)

2 Pip

O gerenciador de pacotes já vem com as versões recentes do Python. Caso necessite instalá-lo separadamente, consulte: [Guia de Instalação do Pip](#)

3 Virtualenv

Para isolar seus projetos e evitar conflitos entre dependências, instale o virtualenv seguindo: [Instruções de Instalação do Virtualenv](#)



Preparando o Ambiente de Desenvolvimento

Verificando a Instalação

Teste o Python: No terminal, digite `python` para abrir o console interativo

```
C:\Users\evert>python
Python 3.10.8 (tags/v3.10.8:aaaf517, Oct 11 2022, 16:50:30) [MSC v.1933 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

Para sair, digite `quit()` ou pressione `CTRL+D`

Verifique o Pip: No terminal, digite `pip --version`

```
C:\Users\evert>pip --version
pip 22.2.2 from C:\Python310\lib\site-packages\pip (python 3.10)
```

 O **pip** é uma ferramenta de linha de comando que significa "Pip Installs Packages". É usado para instalar e gerenciar bibliotecas Python disponíveis no repositório PyPI.

Criando um Ambiente Virtual



Criar

No terminal, digite:
`virtualenv myenv`

Isso cria uma pasta com todos os arquivos necessários



Ativar

Windows: `.\myenv\Scripts\activate.bat`
Linux: `source myenv/bin/activate`



Desativar

Para sair do ambiente virtual, digite:
`deactivate`

```
created virtual environment CPython3.8.6.final.0-64 in 41777ms
creator CPython3Windows(dest=G:\Meu Drive\IFSP - Campinas\TADS\1 - Disciplinas\4-INTI6\1-Aulas\Aula 02 - Ambiente de D
ev - Python\myenv, clear=False, no_vcs_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=C:\Users\evert\
AppData\Local\pypa\virtualenv)
added seed packages: pip==20.3.1, setuptools==50.3.2, wheel==0.36.1
activators BashActivator,BatchActivator,FishActivator,PowerShellActivator,PythonActivator,XonshActivator
```

```
(myenv) G:\Meu Drive\IFSP - Campinas\TADS\1 - Disciplinas\4-INTI6\1-Aulas\Aula 02 - Ambiente de Dev - Python>
```

Uma vez ativado o ambiente virtual, todos os pacotes instalados ficarão contidos apenas neste ambiente, sem afetar a instalação global do Python.

Gerenciando Pacotes Python

Listar Pacotes

Com ambiente virtual ativado:

```
pip list
```

Instalar Pacotes

Com ambiente virtual ativado:

```
pip install nome_do_pacote
```

Desinstalar Pacotes

Com ambiente virtual ativado:

```
pip uninstall nome_do_pacote
```

Pacotes Essenciais

```
pip install ipython  
pip install numpy  
pip install pandas  
pip install sklearn
```

```
pip install jupyterlab  
pip install matplotlib  
pip install seaborn  
pip install plotly
```

Os pacotes serão instalados no diretório: <diretorio_do_ambiente_virtual>/lib/python3.x/site-packages/

Jupyter Lab

O Jupyter Lab é um ambiente web interativo para desenvolver, executar e compartilhar código Python e dados. É a ferramenta ideal para a criação de "notebooks", que combinam código executável, visualizações e texto explicativo.

Principais Vantagens:

1

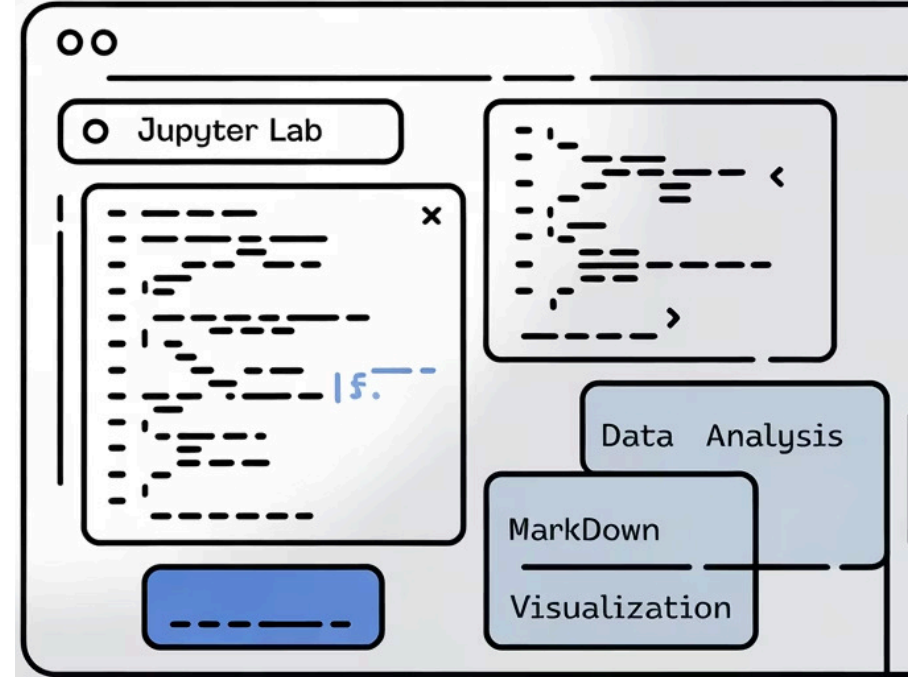
Integração de Código e Texto

Permite alternar fluidamente entre código Python e texto explicativo formatado em Markdown, criando documentos dinâmicos.

2

Ideal para Análise de Dados

Perfeito para análise de dados, experimentos científicos, prototipagem rápida e documentação de projetos de forma colaborativa.



Jupyter Lab

O Jupyter Lab é um ambiente web interativo para desenvolver, executar e compartilhar código Python e dados. Veja como iniciar e seus principais componentes:

Como iniciar seu ambiente de trabalho:



Iniciar o Servidor

No terminal, digite: `jupyter lab` para abrir a interface no seu navegador.



Criar um Novo Notebook

Navegue até a pasta do projeto e crie um novo notebook (.ipynb) para começar a codificar.

Principais Tipos de Células:

Célula de Código

Para escrever e executar comandos Python e ver os resultados imediatamente.

Célula de Markdown

Para adicionar anotações, descrições, fórmulas e explicações formatadas ao seu notebook.

Ambientes Alternativos de Desenvolvimento

“

Google Colab

Uma versão do Jupyter na nuvem do Google, com recursos gratuitos de GPU/TPU.

- Ambiente pronto com principais bibliotecas
- Notebooks salvos no Google Drive
- Compartilhamento facilitado

[Acesse o Google Colab](#)

”



PyCharm

IDE completa e robusta da JetBrains, com recursos avançados para desenvolvimento Python profissional.



Visual Studio Code

Editor de código leve e altamente personalizável com excelentes extensões para Python.

Importante: Configure o VS Code para usar o interpretador Python do seu ambiente virtual!

Fundamentos da Linguagem Python

- **Variáveis e Tipos**

- Case sensitive (nome $\neq$ Nome)
- Atribuição simples: `x = 10`
- Atribuição múltipla: `x, y = 10, 20`
- Linguagem fortemente e dinamicamente tipada
- Conversão de tipos: `int()`, `float()`, `str()`

- **Funções**

- Print: `print("Olá")` ou formatado `print(f"Soma: {x+y}")`
- Definição com `def nome_funcao(parametros):`
- Identação define o escopo da função
- Parâmetros default: `def soma(a=0, b=0):`
- Function Annotations para tipagem

Condicionais

```
if condição:
    # código
elif outra_condição:
    # código
else:
    # código

# Condicional em uma linha
resultado = "par" if numero % 2 == 0 else "ímpar"
```

Funções e Execução de Código

Em Python, as funções são blocos de código reutilizáveis que realizam uma tarefa específica. A execução de um programa Python geralmente começa com um "ponto de entrada" principal.

Exemplo de Função main:

```
def cumprimentar(nome):  
    """Retorna uma saudação personalizada."""  
    return f"Olá, {nome}!"  
  
def main():  
    """Função principal do programa."""  
    print("Iniciando o programa...")  
    msg = cumprimentar("Turma de Aprendizado de Máquina")  
    print(msg)  
    print("Programa finalizado.")  
  
if __name__ == "__main__":  
    main() # Chama a função principal quando o script é executado  
diretamente
```

def main():

Por convenção, a função `main` é usada como o ponto de entrada principal do seu script, organizando a lógica central do programa.

if __name__ == "__main__":

Este bloco de código garante que a função `main()` seja chamada apenas quando o script é **executado diretamente**, e não quando é importado como um módulo em outro script.

Modularidade

O uso de funções promove a modularidade, tornando o código mais legível, fácil de manter e reutilizável em diferentes partes do seu projeto.

Estruturas de Dados e Controle



Loops

```
# Loop com range
for i in range(5):
    print(i) # 0, 1, 2, 3, 4
```

```
# Loop while
contador = 0
while contador < 5:
    print(contador)
    contador += 1
```



Listas

```
# Criação
numeros = [1, 2, 3, 4]
mista = [1, "texto", True]
```

```
# Métodos
numeros.append(5) # Adiciona ao final
numeros.insert(0, 0) # Insere no índice
numeros.pop() # Remove o último
numeros.sort() # Ordena
```

```
# List comprehension
quadrados = [x**2 for x in range(10)]
```

Estruturas de Dados e Controle



Strings

```
texto = "Python é incrível"

# Métodos
maiusculas = texto.upper()
palavras = texto.split()
substituir = texto.replace("incrível", "fantástico")

# Slices
primeiras_letras = texto[:6] # "Python"
```



Slices

Funcionam com listas e strings:

```
lista = [0, 1, 2, 3, 4, 5]
print(lista[1:4]) # [1, 2, 3]
print(lista[::2]) # [0, 2, 4]
print(lista[::-1]) # [5, 4, 3, 2, 1, 0]
```

Estruturas de Dados Avançadas

Tuplas

```
# Criação
vazia = ()
numeros = (1, 2, 3)
mista = (1, "texto", True)

# Imutabilidade
# numeros[0] = 5 # Erro!

# Conversão
lista = list(numeros)
tupla = tuple(lista)

# Acesso
primeiro = numeros[0]
```



Dicionários

```
# Criação
usuario = {
    "nome": "Ana",
    "idade": 25,
    "admin": False
}

# Acesso e modificação
print(usuario["nome"])
usuario["email"] = "ana@email.com"

# Iteração
for chave, valor in usuario.items():
    print(f"{chave}: {valor}")
```

Estruturas de Dados Avançadas

Módulos e Comentários

```
# Importação de módulos
import math
from random import randint
```

```
# Comentário de linha única
```

```
"""
```

Docstring: Documentação de múltiplas linhas para funções, classes e módulos

```
"""
```

```
def calcular_area(raio):
    """Calcula a área de um círculo."""
    return math.pi * raio ** 2
```

```
# Código principal
```

```
if __name__ == "__main__":
    print("Este arquivo está sendo executado diretamente")
    print(f"Área do círculo: {calcular_area(5)}")
```



Tópicos Avançados



Manipulação de Arquivos

```
# Leitura de arquivo
with open('arquivo.txt', 'r') as f:
    conteudo = f.read()

# Escrita em arquivo
with open('saida.txt', 'w') as f:
    f.write('Olá, Python!')
```



Pacotes

Organização de módulos em diretórios com
`__init__.py`

```
# Estrutura de um pacote
meu_pacote/
__init__.py
modulo1.py
modulo2.py
```

```
# Importação
from meu_pacote import modulo1
```



Classes e Objetos

```
class Pessoa:
    def __init__(self, nome, idade):
        self.nome = nome
        self.idade = idade

    def saudacao(self):
        return f"Olá, meu nome é {self.nome}"

# Criação de objeto
pessoa = Pessoa("Maria", 30)
print(pessoa.saudacao())
```

Estes tópicos formam a base para o desenvolvimento de aplicações mais complexas em Python, incluindo sistemas web, análise de dados e inteligência artificial.

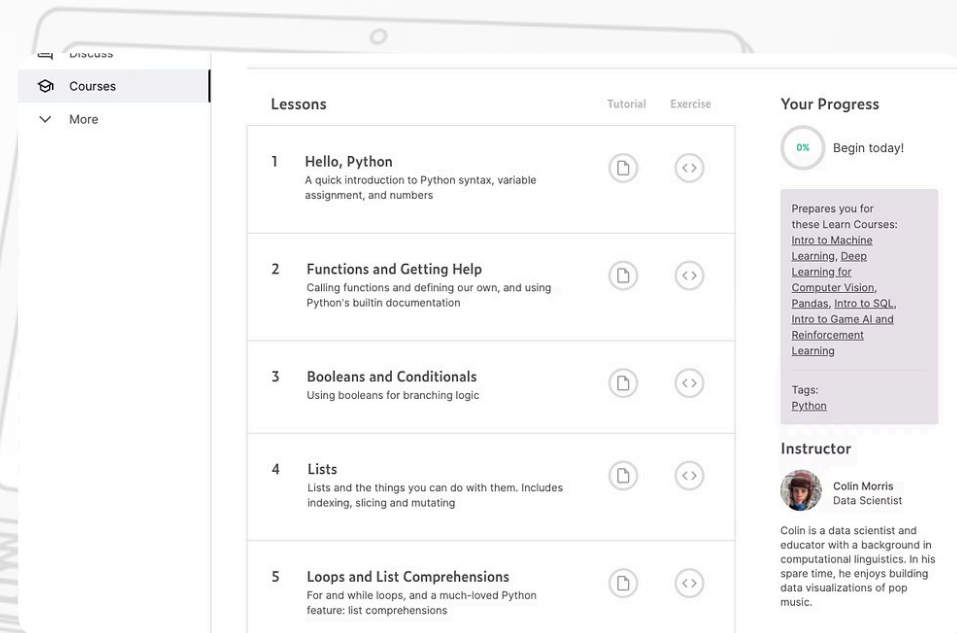
Recursos Adicionais



Curso em Vídeo - Python 3

Curso completo e gratuito em português ministrado pelo professor Gustavo Guanabara.

[Acessar o curso](#)



Kaggle - Python

Aprenda Python enquanto resolve problemas reais de análise de dados e participe de competições.

[Acessar o curso no Kaggle](#)

 Mantenha-se praticando constantemente e participando da comunidade Python para aprimorar suas habilidades!